

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	I.Guru Hemanth Reddt	Reg. No.:	192311222
Date:	3/08/2026	Duration:	60 minutes

Code of Conduct

I Hemanth Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Hemanth

Annexure A – Scenario-Based Requirement Analysis

Smart College Bus Tracking and Management System

Table of Contents

1	Table of Contents	Error! Bookmark not defined.
2	SECTION 1 – STUDY AND ANALYZE THE SCENARIO	3
2.1	1.1 Business Domain	3
2.2	1.2 Existing System	3
2.3	1.3 Current Challenges	3
2.4	1.4 Proposed Solution	3
2.5	1.5 System Objectives	4
2.6	1.6 Stakeholders	4
2.7	1.7 Business Goals	4
3	SECTION 2 – FUNCTIONAL REQUIREMENTS	4
4	SECTION 3 – NON-FUNCTIONAL REQUIREMENTS	5
5	SECTION 4 – ACTORS AND USE CASES	5
5.1	4.1 Primary Actors	5
5.2	4.2 Secondary Actors	5

5.3	4.3 Use Case Descriptions	5
6	SECTION 5 – SOFTWARE REQUIREMENT SPECIFICATION (IEEE 830)	6
6.1	1. Introduction	6
6.1.1	1.1 Purpose	6
6.2	2. Overall Description.....	6
6.2.1	2.1 Product Perspective	6
6.3	3. Specific Requirements	6
7	SECTION 6 – DATA REQUIREMENTS	7
7.1	6.1 Data Dictionary	7
7.2	6.2 Database Tables.....	7
8	SECTION 7 – EXTERNAL INTERFACE REQUIREMENTS.....	7
8.1	7.1 User Interface	7
8.2	7.2 Hardware Interface	7
8.3	7.3 Software Interface.....	7
8.4	7.4 Communication Interface.....	7
8.5	7.5 API Integration	7
9	SECTION 8 – SYSTEM FEATURES	7
9.1	8.1 Authentication Module	7
9.2	8.2 Student Module	8
9.3	8.3 Driver Module	8
9.4	8.4 Admin Module	8
9.5	8.5 GPS Tracking Engine	8
9.6	8.6 Notifications Engine	8
9.7	8.7 Reports and Analytics	8
10	SECTION 9 – ASSUMPTIONS	8
10.1	9.1 Business Assumptions	8
10.2	9.2 Technical Assumptions.....	8
10.3	9.3 Operational Assumptions.....	8
11	SECTION 10 – CONSTRAINTS	8
11.1	10.1 Technical Constraints.....	8
11.2	10.2 Economic Constraints	9
11.3	10.3 Legal Constraints	9
11.4	10.4 Operational Constraints.....	9
11.5	10.5 Schedule Constraints.....	9
11.6	10.6 Resource Constraints	9
12	SECTION 11 – REQUIREMENT VALIDATION	9

12.1	12.1 System Architecture Diagram	9
12.2	12.2 Use Case Diagram	10
12.3	12.3 Activity Diagram	11
12.4	12.4 Sequence Diagram	12
12.5	12.5 Class Diagram	13
12.6	12.6 ER Diagram.....	15
12.7	12.7 DFD Level 0 (Context Diagram).....	17
12.8	12.8 DFD Level 1	17
12.9	12.9 State Diagram	17
12.10	12.10 Deployment Diagram	18
13	SECTION 13 – APPENDIX.....	19
13.1	13.1 Glossary	19
13.2	13.2 Abbreviations	19
13.3	13.3 References	19
13.4	13.4 IEEE References.....	19

2 SECTION 1 – STUDY AND ANALYZE THE SCENARIO

2.1 1.1 Business Domain

The business domain lies in Educational Logistics and Intelligent Transportation Systems (ITS). Educational institutions manage large fleets of buses to transport students and faculty across wide geographic areas. Efficient management of this fleet directly impacts operational costs, student safety, and institutional reputation.

2.2 1.2 Existing System

Currently, the college uses a manual scheduling and routing system. Students rely on printed timetables or word-of-mouth to track buses. Drivers use physical logbooks for attendance and fuel tracking. Communication between the transport admin and the drivers is conducted via phone calls.

2.3 1.3 Current Challenges

- **Unpredictability:** Students wait for extended periods at stops with no visibility into bus locations or delays.
- **Inefficiency:** Routes are statically defined and do not dynamically adjust to traffic or passenger load.
- **Safety Concerns:** In the event of breakdowns or emergencies, communication is slow.
- **Lack of Analytics:** Admins have no real-time data on vehicle speed, fuel consumption, or driver adherence to schedules.

2.4 1.4 Proposed Solution

The “Smart College Bus Tracking and Management System” is a centralized cloud platform with integrated mobile applications. Drivers will use a mobile app to transmit real-time GPS coordinates. Students and parents will use an app to view live tracking, ETAs, and receive proximity alerts. Administrators will use a web dashboard for complete fleet oversight and analytics.

2.5 1.5 System Objectives

- To provide real-time GPS tracking with less than 5 seconds of latency.
- To reduce student wait times by providing accurate ETAs and push notifications.
- To optimize fleet operations by analyzing route efficiency and fuel consumption.
- To enhance safety through SOS alerts and driver monitoring.

2.6 1.6 Stakeholders

- **Students/Faculty:** Primary end-users tracking buses.
- **Parents:** Secondary users tracking student safety.
- **Drivers:** App users transmitting location data.
- **Transport Administrators:** Managers monitoring the fleet.
- **College Management:** Executive sponsors.

2.7 1.7 Business Goals

- Increase student satisfaction by 80% regarding transport reliability.
- Reduce fuel consumption by 15% through optimized routing.
- Eliminate manual logbook errors by digitizing all tracking processes.

3 SECTION 2 – FUNCTIONAL REQUIREMENTS

Req ID	Requirement	Description	Priority	Actor
FR01	User Registration	Register using university ID	High	Student, Driver
FR02	Secure Login	Login with password or OTP	High	All Users
FR03	Role-Based Access	Access based on user role	High	Admin
FR04	GPS Tracking	Send GPS location every 5 sec	High	Driver
FR05	Live Bus Tracking	View live bus location	High	Student, Parent
FR06	Route Selection	Select assigned bus route	High	Student
FR07	ETA Display	Display estimated arrival time	High	Student
FR08	Notifications	Notify when bus is nearby	High	Student
FR09	SOS Alert	Send emergency alert	High	Driver
FR10	Route Management	Create and manage routes	Medium	Admin
FR11	Driver Assignment	Assign drivers to buses	High	Admin
FR12	Fleet Monitoring	Monitor all active buses	High	Admin
FR13	Delay Alerts	Send delay notifications	Medium	Admin
FR14	Speed Monitoring	Detect over-speeding	Medium	Admin
FR15	Trip History	View previous trips	Low	Driver
FR16	Leave Request	Submit leave requests	Low	Driver
FR17	Stop Management	Add or update bus stops	Medium	Admin
FR18	Favorite Stops	Save favorite stops	Low	Student

Req ID	Requirement	Description	Priority	Actor
FR19	Parent Linking	Link parent and student	Medium	Parent
FR20	QR Attendance	Record boarding using QR	Medium	Student
FR21	Report Generation	Export reports	Medium	Admin
FR22	Language Support	Support multiple languages	Low	All Users
FR23	Feedback	Submit feedback	Low	Student
FR24	Password Reset	Reset password	High	All Users
FR25	Session Timeout	Auto logout after inactivity	Medium	Admin

4 SECTION 3 – NON-FUNCTIONAL REQUIREMENTS

Category	Requirement Details
Performance	Support up to 10,000 concurrent users with response time under 2 seconds.
Security	Encrypt all data using TLS 1.3 and store passwords using bcrypt hashing.
Reliability	Ensure 99.9% system uptime with minimal downtime.
Availability	System shall be available 24×7 except scheduled maintenance.
Scalability	Support increasing users and data through scalable cloud infrastructure.
Maintainability	Use modular code, coding standards, and proper documentation.
Portability	Support Android, iOS, Windows, Linux, and modern web browsers.
Usability	Provide a simple, user-friendly interface accessible within three clicks.
Compatibility	Integrate with the college database and external APIs using REST services.
Backup & Recovery	Perform daily backups and restore data within one hour if required.
Data Integrity	Ensure data accuracy and consistency using database constraints and ACID properties.
Privacy	Protect user information and store only essential data according to privacy policies.

5 SECTION 4 – ACTORS AND USE CASES

5.1 4.1 Primary Actors

- **Student:** Registers, views routes, tracks bus, receives alerts.
- **Parent:** Links to student profile, tracks specific bus for safety.
- **Driver:** Logs in, starts trip, transmits GPS, triggers SOS.
- **Transport Administrator:** Manages fleet, assigns drivers, views analytics, broadcasts alerts.

5.2 4.2 Secondary Actors

- **GPS Service (e.g., Google Maps API):** Provides mapping, routing, and reverse geocoding data.
- **Notification Service (e.g., Firebase Cloud Messaging):** Dispatches push notifications to devices.
- **Database (e.g., PostgreSQL):** Stores permanent user and route data.
- **Authentication Server:** Handles OAuth tokens and session management.

5.3 4.3 Use Case Descriptions

Use Case 1: View Live Tracking * **Purpose:** Allow students to see bus location. * **Preconditions:** Student is logged in and route is active. * **Main Flow:** Student selects route → System fetches GPS → Map renders bus. * **Alternative Flow:** If GPS is lost, system displays “Signal Lost - Last known location.” * **Postconditions:** Student receives live updates.

Use Case 2: Transmit Location * **Purpose:** Driver sends live coordinates to server. * **Preconditions:** Driver has started the trip. * **Main Flow:** App accesses device GPS → App sends HTTP POST with coordinates every 5s. * **Alternative Flow:** If network drops, app caches coordinates and batch-sends upon reconnection. * **Postconditions:** Database is updated with the latest coordinates.

Use Case 3: Manage Routes * **Purpose:** Admin creates or modifies bus paths. * **Preconditions:** Admin is authenticated. * **Main Flow:** Admin opens map → Draws polyline → Adds stops → Saves. * **Alternative Flow:** Admin selects existing route → modifies waypoint → updates. * **Postconditions:** Route database is updated and synced to drivers.

Use Case 4: Trigger SOS * **Purpose:** Driver alerts admin of emergency. * **Preconditions:** Trip is active. * **Main Flow:** Driver holds SOS button for 3 seconds → System alerts Admin dashboard with siren and red flag. * **Alternative Flow:** Accidental press (under 3s) is ignored. * **Postconditions:** Admin dispatches help.

6 SECTION 5 – SOFTWARE REQUIREMENT SPECIFICATION (IEEE 830)

6.1 1. Introduction

6.1.1 1.1 Purpose

This SRS describes the software functional and non-functional requirements for release 1.0 of the Smart College Bus Tracking System. #### 1.2 Scope The system will replace manual bus tracking. It covers a mobile application for tracking and a web application for administration. #### 1.3 Definitions * **ETA:** Estimated Time of Arrival. * **GPS:** Global Positioning System. #### 1.4 References * IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications. #### 1.5 Overview The rest of this document contains the overall description, specific requirements, data requirements, external interfaces, and diagrams defining the architecture.

6.2 2. Overall Description

6.2.1 2.1 Product Perspective

The system is a self-contained, new product that interfaces with existing university authentication servers and third-party mapping APIs. #### 2.2 Product Functions Live tracking, ETA calculation, route management, fleet analytics, and emergency alerts. #### 2.3 User Classes Students (tech-savvy), Drivers (requires simple UI), Admins (requires detailed analytics). #### 2.4 Operating Environment Mobile (Android/iOS) and Web (Chrome, Safari, Firefox). #### 2.5 Design Constraints Must use lightweight data formats (JSON) to minimize mobile data usage for students. #### 2.6 User Documentation In-app tutorials for drivers; web manuals for administrators. #### 2.7 Assumptions Drivers have smartphones with active data plans. #### 2.8 Dependencies Relies strictly on the uptime of the Google Maps API and Firebase Cloud Messaging.

6.3 3. Specific Requirements

(Covered comprehensively in Sections 2, 3, 6, 7, and 8 of this document).

7 SECTION 6 – DATA REQUIREMENTS

7.1 6.1 Data Dictionary

- **User_ID:** Unique alphanumeric identifier for each user.
- **Coordinates:** GPS data stored as floats (Latitude, Longitude).
- **Timestamp:** UNIX epoch time for tracking events.
- **Route_ID:** Unique identifier mapping a bus to a physical path.

7.2 6.2 Database Tables

Table: Users | Attribute | Type | Primary Key | Foreign Key | Validation Rules | | :— | :— | :— | :— | :—
| | UserID | VARCHAR | Yes | No | Not Null, Unique | | Role | ENUM | No | No | ‘Student’, ‘Admin’,
‘Driver’ | | PasswordHash | VARCHAR | No | No | Min 64 chars (bcrypt) |

Table: Routes | Attribute | Type | Primary Key | Foreign Key | Validation Rules | | :— | :— | :— | :— | :—
:— | | RouteID | VARCHAR | Yes | No | Not Null | | DriverID | VARCHAR | No | Yes (Users.UserID) |
Must exist in Users where Role=‘Driver’ | | Polyline | TEXT | No | No | Valid Google Maps encoded
polyline |

Table: LiveTracking | Attribute | Type | Primary Key | Foreign Key | Validation Rules | | :— | :— | :— | :— | :—
:— | :— | | TrackingID | INT | Yes | No | Auto Increment | | RouteID | VARCHAR | No | Yes
(Routes.RouteID) | Not Null | | Latitude | FLOAT | No | No | Between -90 and 90 | | Longitude | FLOAT |
No | No | Between -180 and 180 | | Timestamp | DATETIME | No | No | Not Future Date |

8 SECTION 7 – EXTERNAL INTERFACE REQUIREMENTS

8.1 7.1 User Interface

- **Student App:** Clean, modern interface with a large map view occupying 70% of the screen. Bottom sheet for ETA and route details.
- **Admin Web Dashboard:** Sidebar navigation. Main view contains a massive interactive map, charts built with Chart.js for analytics, and data tables for user management.

8.2 7.2 Hardware Interface

- **Mobile Devices:** Utilizes built-in GPS modules and Accelerometers.
- **RFID Scanners (Optional):** Integration via USB/Bluetooth to the driver’s tablet for student attendance scanning.

8.3 7.3 Software Interface

- **Operating Systems:** Android, iOS, Windows, macOS.
- **Web Browsers:** Google Chrome v90+, Safari v14+, Edge.

8.4 7.4 Communication Interface

- **WebSockets (WSS):** For bi-directional, low-latency transmission of GPS coordinates.
- **HTTPS (TLS 1.3):** For all standard REST API calls (login, route fetching).

8.5 7.5 API Integration

- **Google Maps API:** Used for mapping, polyline rendering, and traffic-aware ETA calculations.
- **Firebase Cloud Messaging (FCM):** Used for dispatching push notifications.
- **University Auth API:** Used for single sign-on (SSO) verification of student credentials.

9 SECTION 8 – SYSTEM FEATURES

9.1 8.1 Authentication Module

Manages secure login, registration, password resets, and session management using JWT (JSON Web Tokens).

9.2 8.2 Student Module

Allows students to select routes, view live maps, pin favorite stops, and configure notification proximity thresholds.

9.3 8.3 Driver Module

A highly simplified, distraction-free UI featuring a massive “Start Trip” button, an SOS button, and a background service that pings the GPS sensor continuously.

9.4 8.4 Admin Module

The core control panel. Allows CRUD operations on users and routes. Handles assigning drivers to specific buses.

9.5 8.5 GPS Tracking Engine

The backend microservice built on Node.js/Socket.io that receives thousands of coordinate pings per second and broadcasts them to subscribed student clients.

9.6 8.6 Notifications Engine

A cron-job and event-driven worker that calculates the distance between buses and stops, triggering Firebase alerts when the 2km threshold is breached.

9.7 8.7 Reports and Analytics

Aggregates historical data to generate CSV reports on route efficiency, average speeds, and habitual delays.

10 SECTION 9 – ASSUMPTIONS

10.1 9.1 Business Assumptions

- The university transport department will actively use the dashboard to update routes rather than reverting to manual scheduling.
- Students will adopt the app as their primary source of truth for transport.

10.2 9.2 Technical Assumptions

- Drivers possess smartphones with functioning GPS chips.
- The campus and surrounding city have adequate 4G/LTE cellular coverage to maintain WebSocket connections.

10.3 9.3 Operational Assumptions

- Drivers will remember to press “Start Trip” before departing the depot. (A geofence trigger acts as a fallback).

11 SECTION 10 – CONSTRAINTS

11.1 10.1 Technical Constraints

- The system must minimize battery drain on the driver’s phone, necessitating optimized GPS polling algorithms.

- Google Maps API requests must be batched or cached to avoid exceeding free-tier limits.

11.2 10.2 Economic Constraints

- The budget for monthly cloud hosting (AWS/GCP) is capped; thus, serverless architecture will be used to scale down to zero during nighttime.

11.3 10.3 Legal Constraints

- Must comply with data protection laws (e.g., GDPR). Student locations cannot be tracked; only bus locations are tracked.

11.4 10.4 Operational Constraints

- The system must not distract the driver. The driver app UI must lock while the vehicle is in motion (except for the SOS button).

11.5 10.5 Schedule Constraints

- The MVP must be completed and deployed within 3 months, before the new academic semester begins.

11.6 10.6 Resource Constraints

- Developed by a small internal engineering team without dedicated QA testers, requiring extensive automated unit testing.

12 SECTION 11 – REQUIREMENT VALIDATION

- **Requirement Completeness:** Verified by tracing all initial stakeholder interviews to specific functional requirements in Section 2.
- **Requirement Consistency:** Verified to ensure no contradictory rules exist (e.g., Drivers cannot be Admins simultaneously).
- **Requirement Correctness:** Approved by the Head of Transport Operations via sign-off.
- **Requirement Traceability:** Each FR is mapped to a specific Use Case (e.g., FR04 maps to Use Case 2).
- **Requirement Feasibility:** Technical feasibility confirmed via a proof-of-concept WebSocket server built during Sprint 0.
- **Requirement Ambiguity:** Removed subjective terms like “fast” and replaced them with quantitative metrics (e.g., “under 5 seconds”).

Suggestions for Improvement: * Future iterations should include ML-based predictive ETAs rather than relying solely on standard mapping APIs. * Integration with physical IoT GPS trackers wired to the bus battery would eliminate dependency on the driver’s personal smartphone battery. # SECTION 12 – DIAGRAMS

12.1 12.1 System Architecture Diagram

The following diagram illustrates the high-level architecture of the Smart College Bus Tracking System, showing the interaction between mobile clients, the API gateway, microservices, and databases.

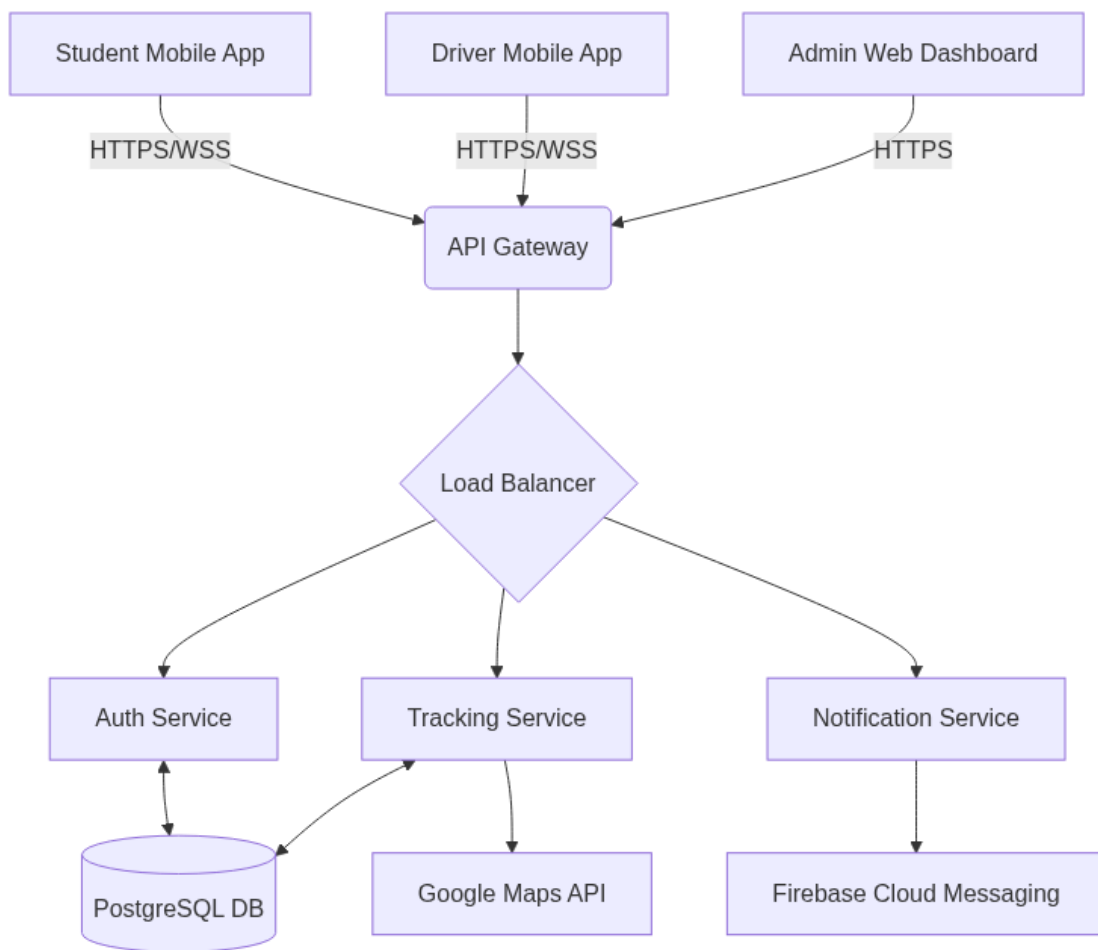


Figure 1:

System Architecture Diagram

12.2 12.2 Use Case Diagram

Maps the primary actors to their respective capabilities within the system.

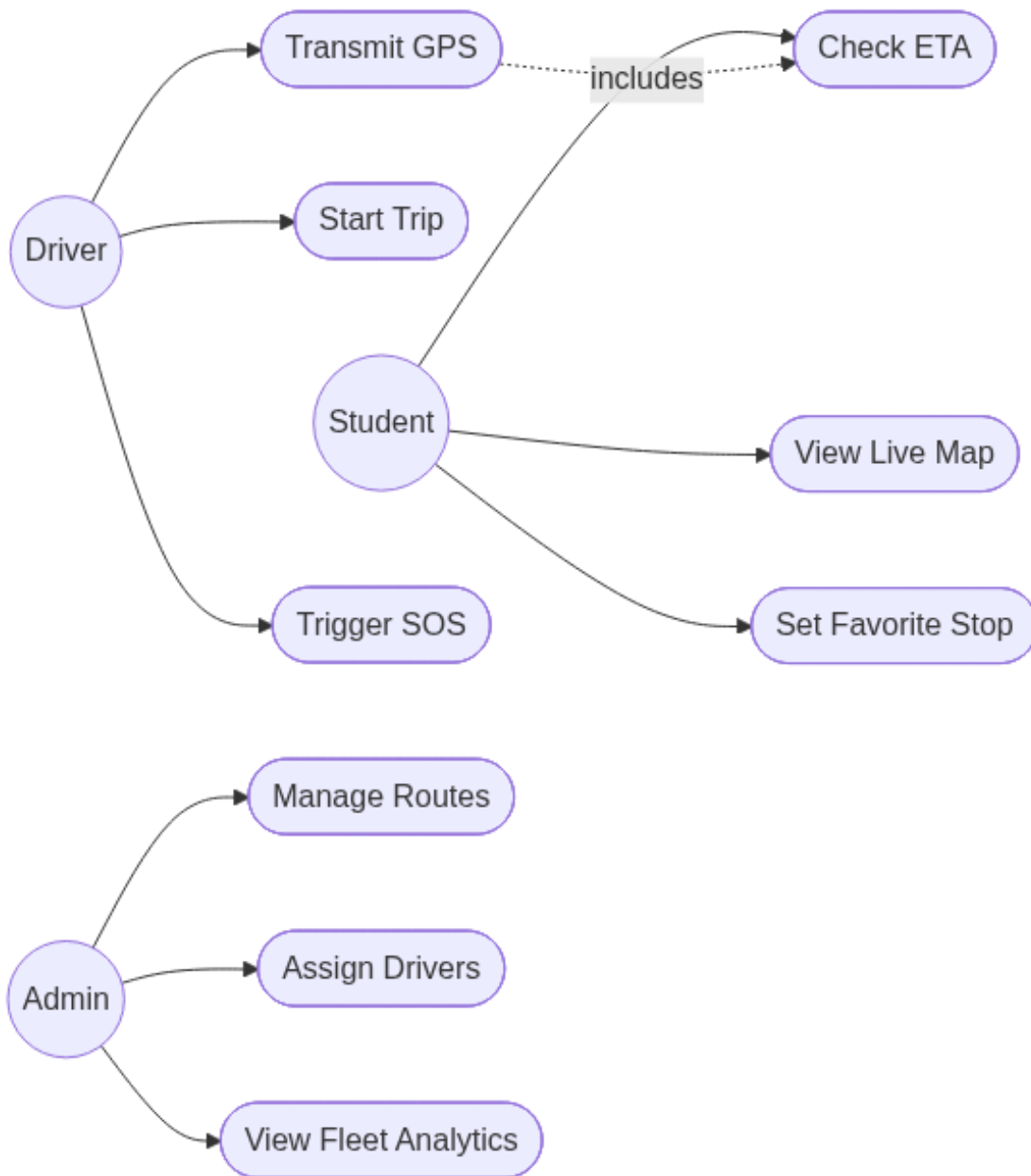


Figure 2: Use

Case Diagram

12.3 12.3 Activity Diagram

Illustrates the flow of a driver starting a trip and the system tracking it.

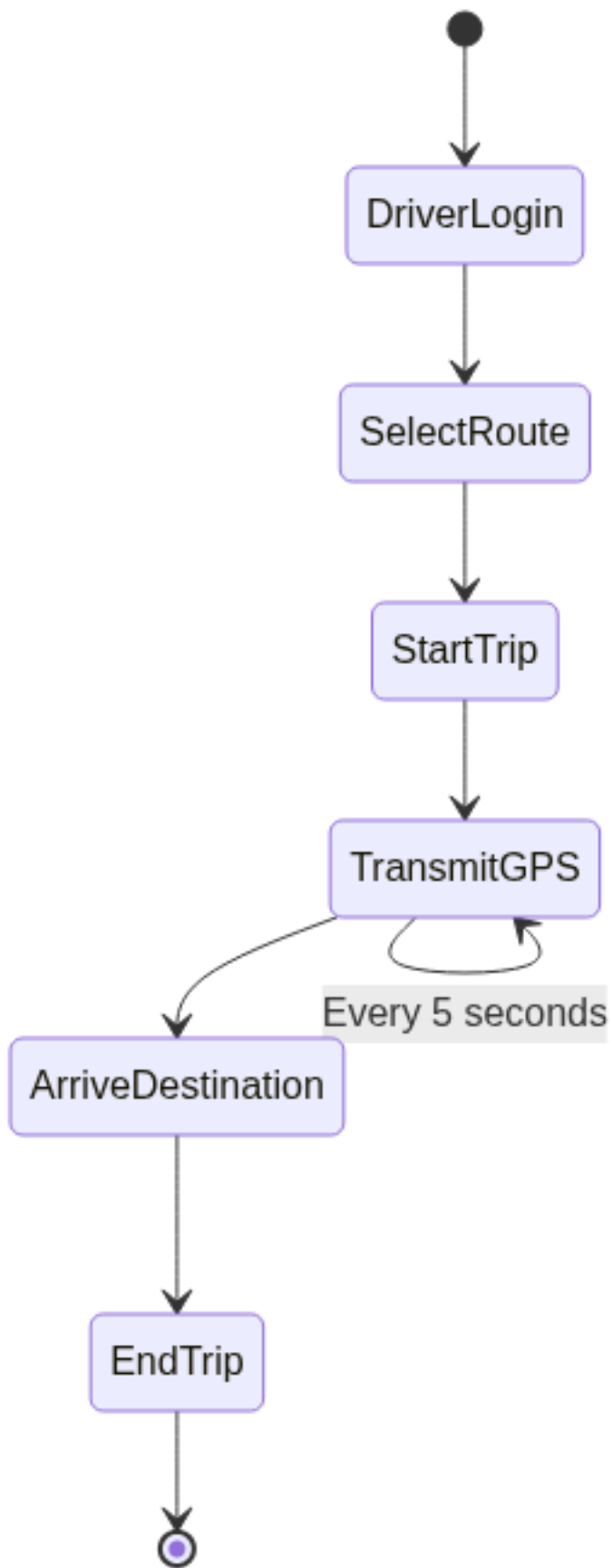


Figure 3: Activity Diagram

Shows the chronological sequence of messages when a student requests an ETA.

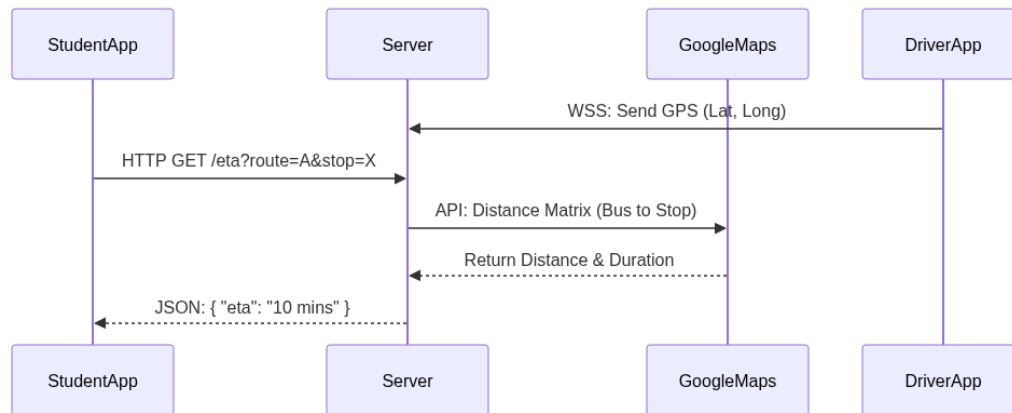
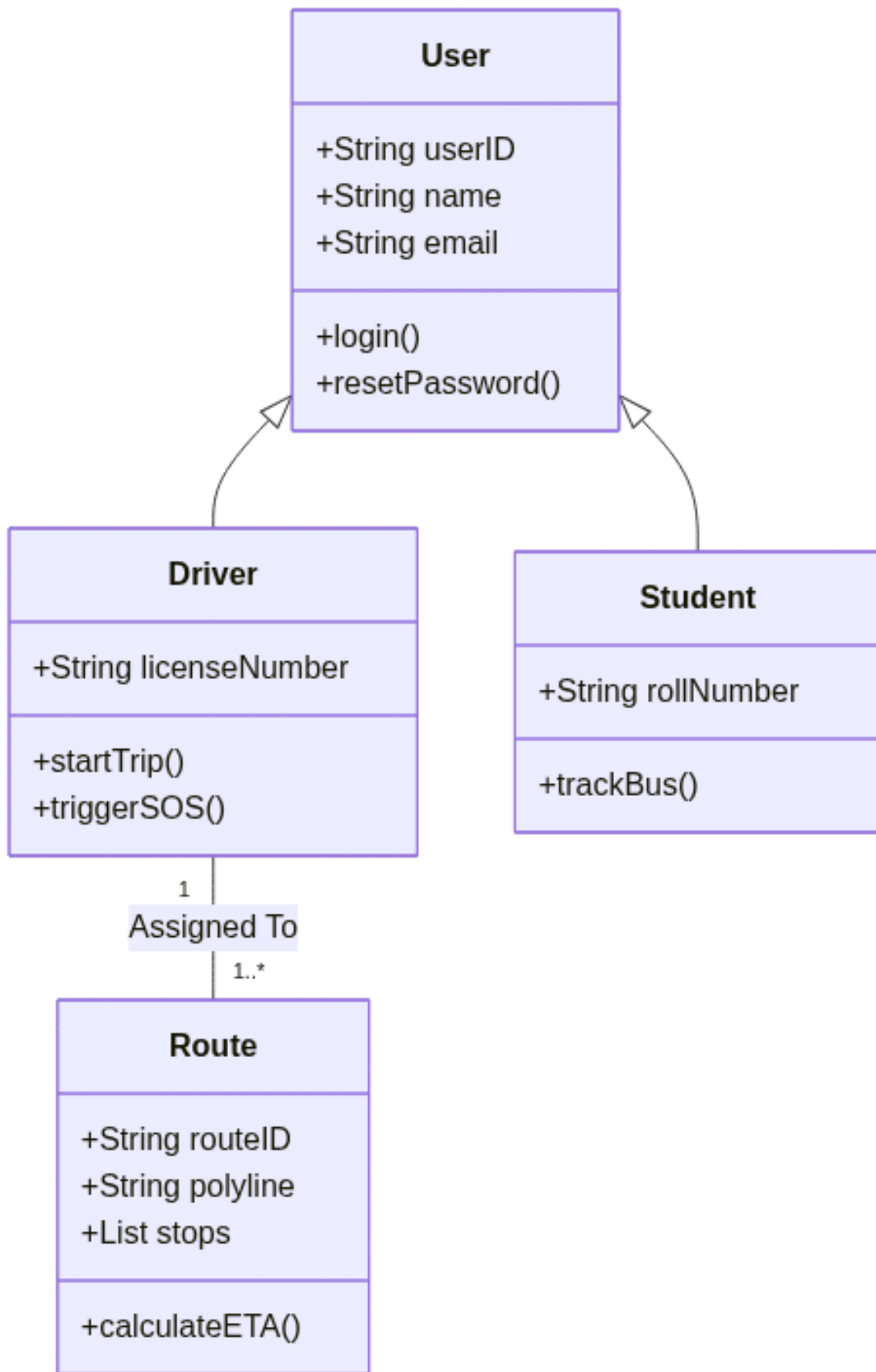


Figure 4:

Sequence Diagram

12.5 12.5 Class Diagram

Details the core object-oriented structures and relationships.

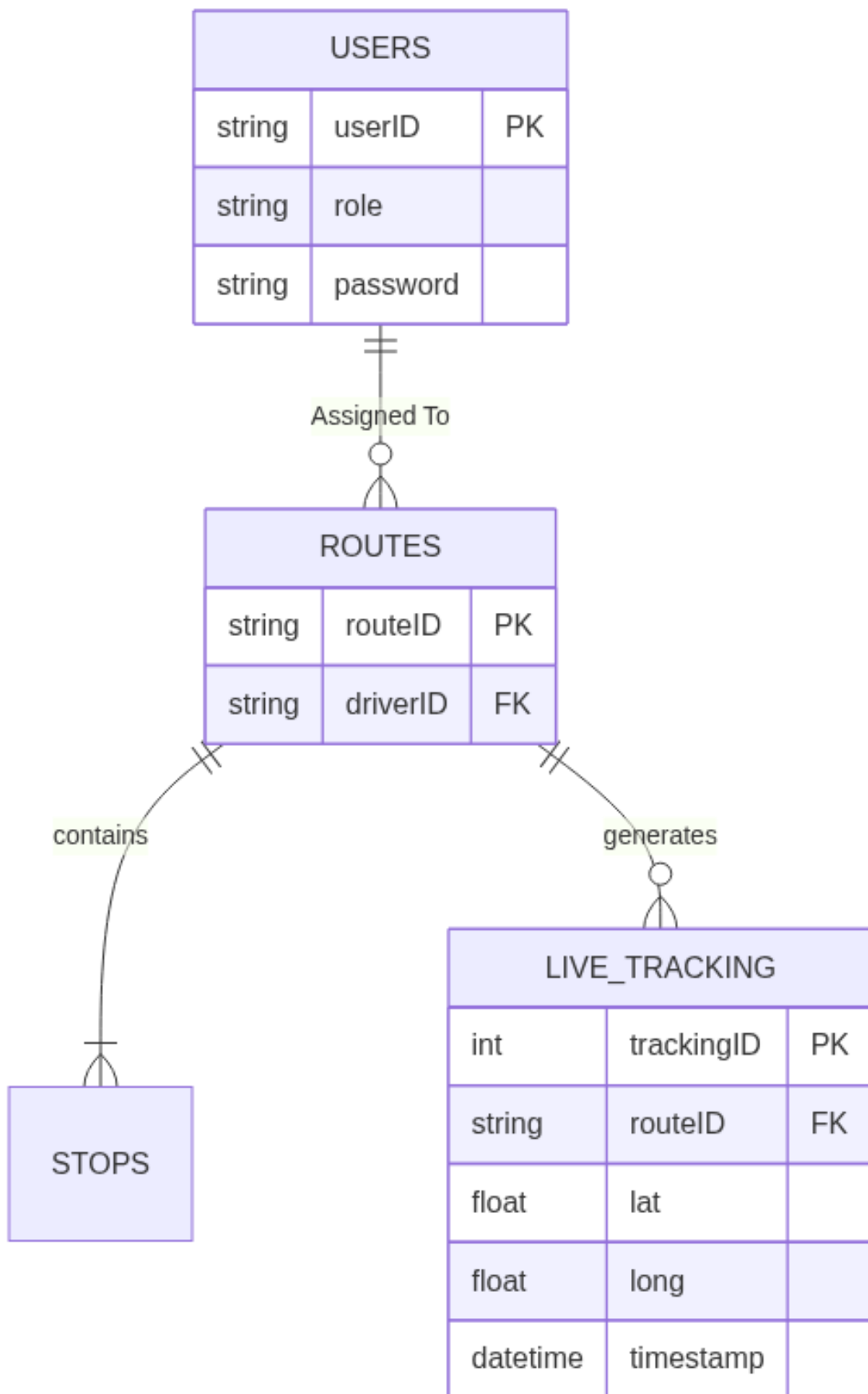


Diagram

Figure 5: Class

12.6 12.6 ER Diagram

Visualizes the database schema and entity relationships.



Relationship Diagram

Figure 6: Entity

12.7 12.7 DFD Level 0 (Context Diagram)

The highest level view of the system interacting with external entities.

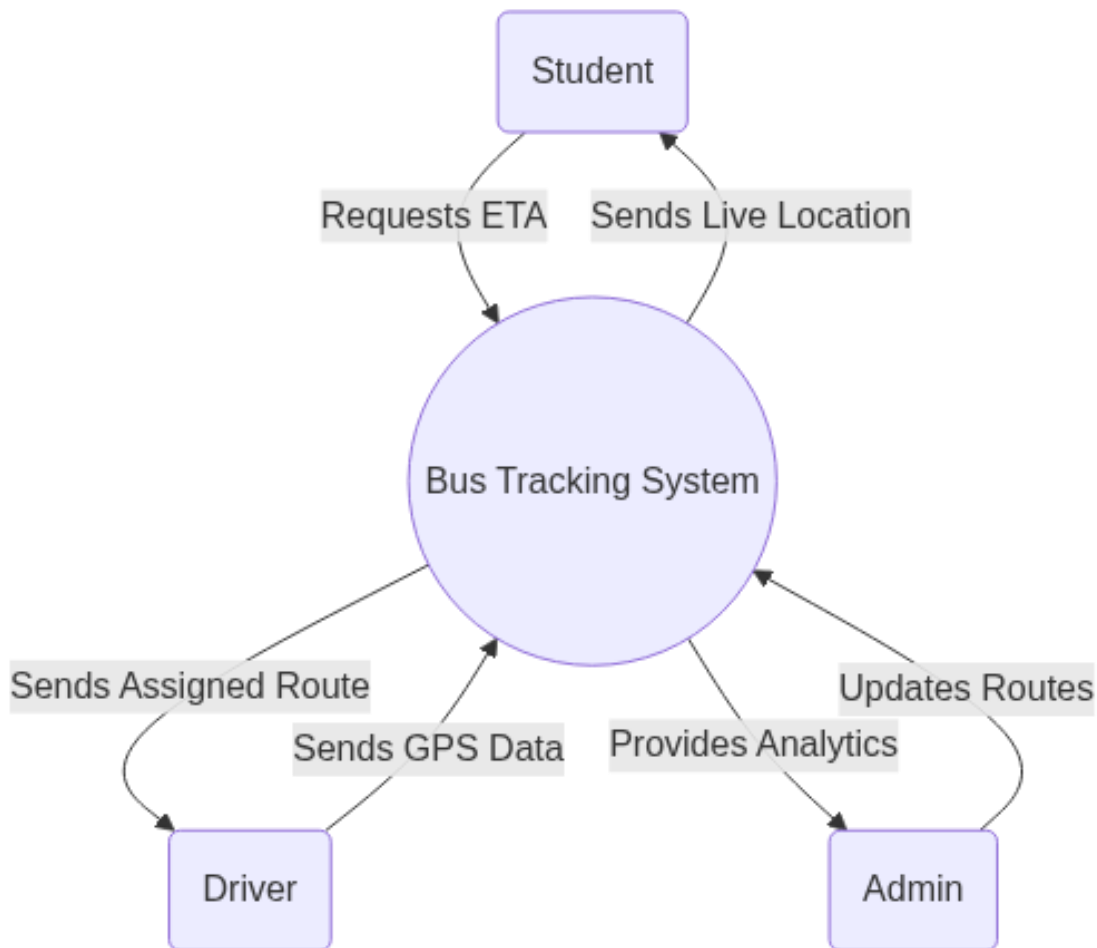


Figure 7: DFD

Level 0

12.8 12.8 DFD Level 1

Breaks down the context diagram into sub-processes.

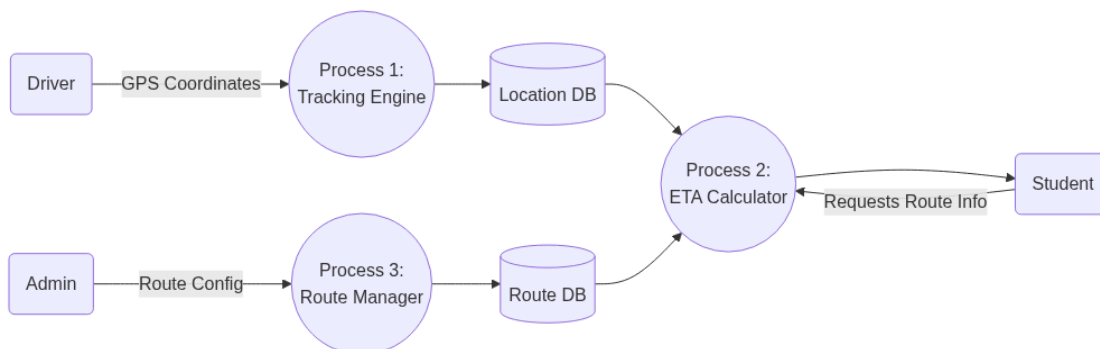


Figure 8: DFD

Level 1

12.9 12.9 State Diagram

Represents the lifecycle state machine of a Bus Trip.

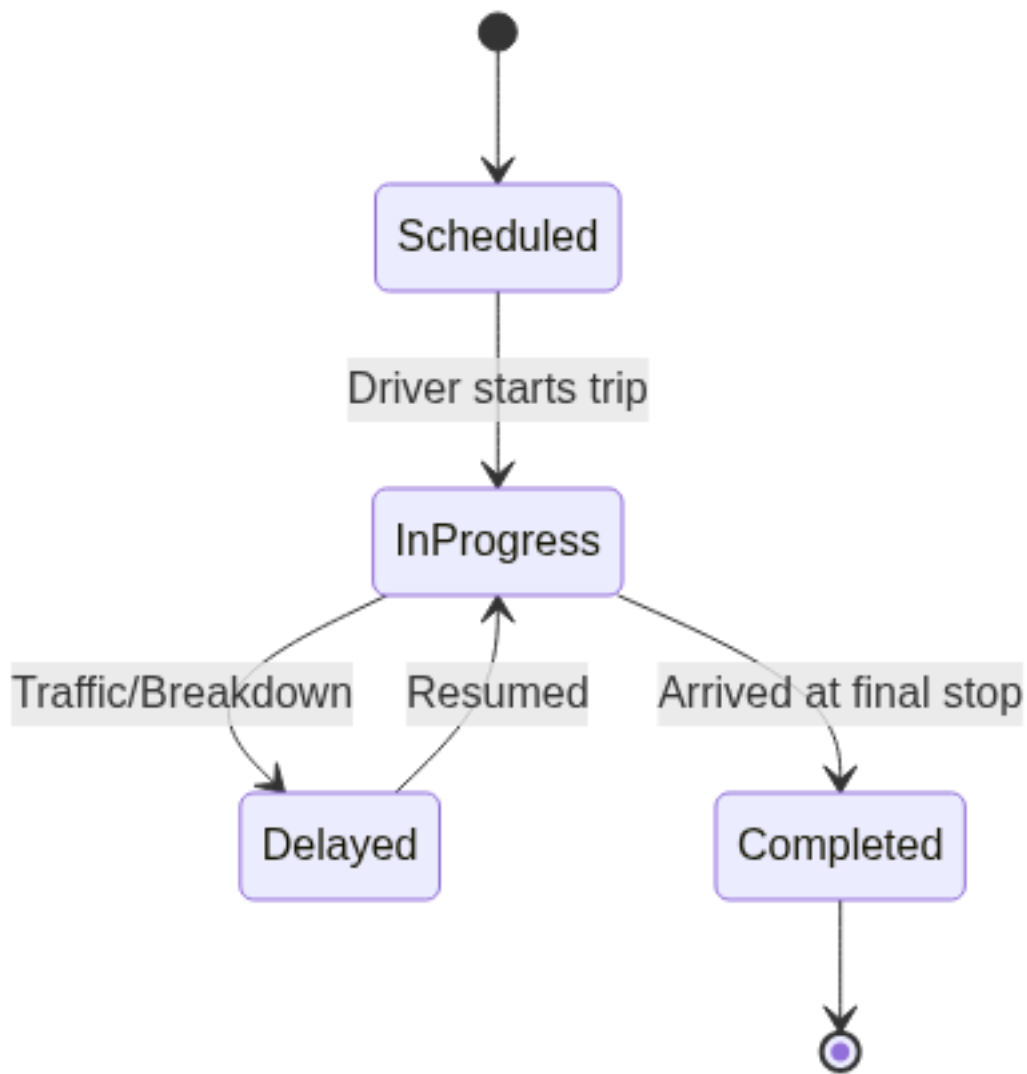


Figure 9: State

Diagram

12.10 12.10 Deployment Diagram

Shows the physical deployment of software nodes on hardware.

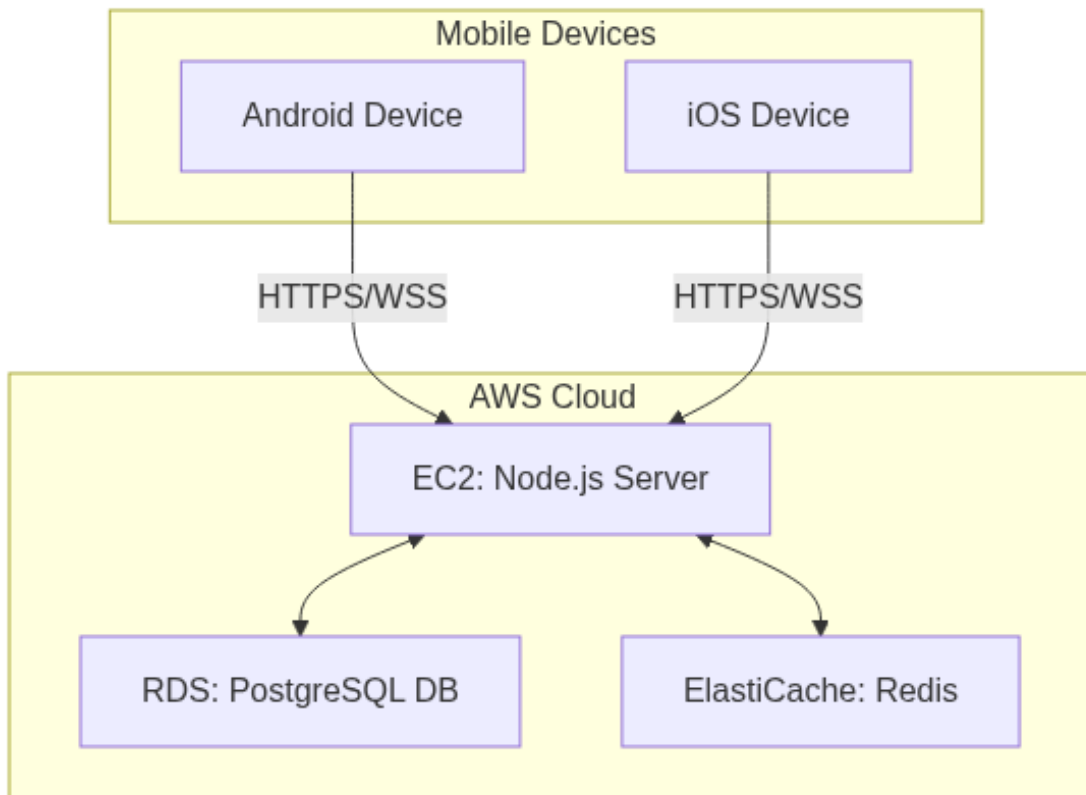


Figure 10:

Deployment Diagram

13 SECTION 13 – APPENDIX

13.1 13.1 Glossary

- **WSS:** WebSocket Secure. A protocol providing full-duplex communication channels over a single TCP connection.
- **Polyline:** A continuous line composed of one or more line segments, used to draw routes on a map.
- **Geofence:** A virtual geographic boundary defined by GPS technology.

13.2 13.2 Abbreviations

- **SRS:** Software Requirement Specification
- **ETA:** Estimated Time of Arrival
- **API:** Application Programming Interface
- **FCM:** Firebase Cloud Messaging
- **JSON:** JavaScript Object Notation

13.3 13.3 References

- [1] Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
- [2] Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

13.4 13.4 IEEE References

- [3] IEEE Computer Society. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers.